Types of Machine Learning

Supervised — data points have known outcome

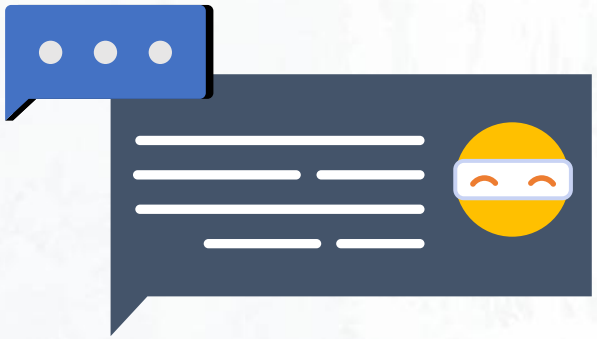
Unsupervised — data points have unknown outcome

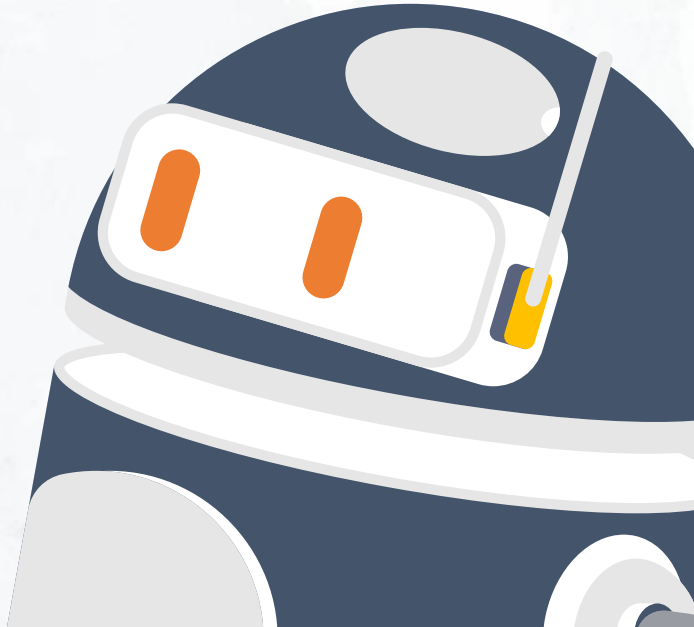
Types of Supervised Learning

Regression

outcome is continuous (numerical)

Linear/Logistic Regression





(AI)

Table of contents

01 → Linear Regression

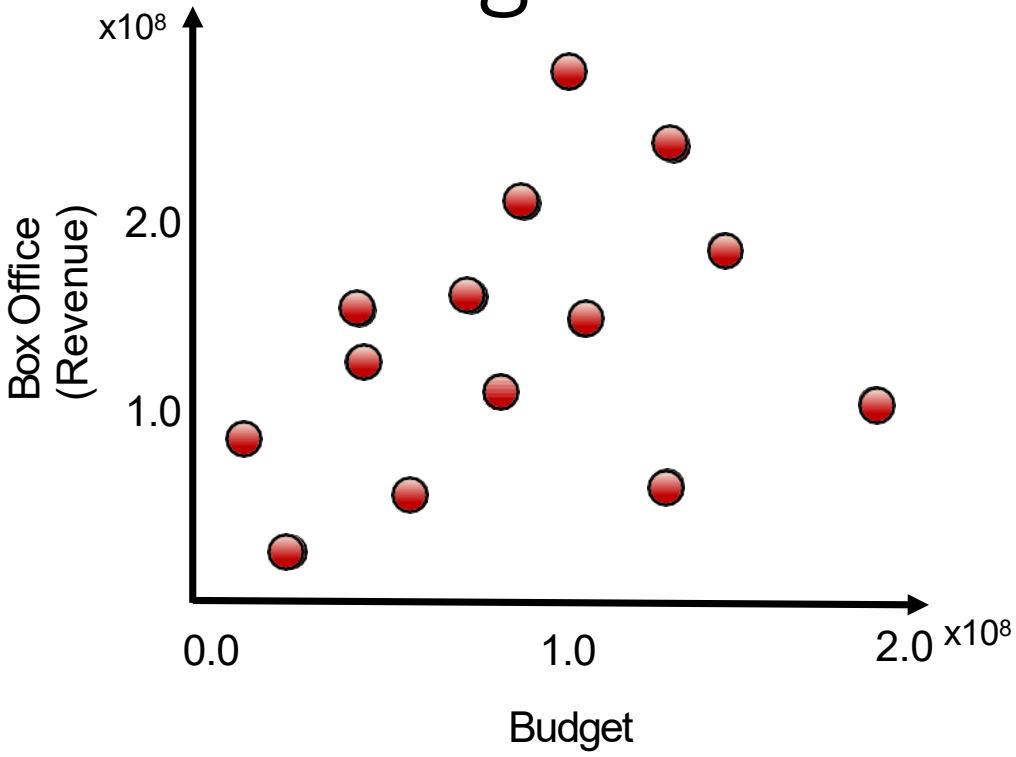
02 → Logistic Regression

01 →

Linear Regression

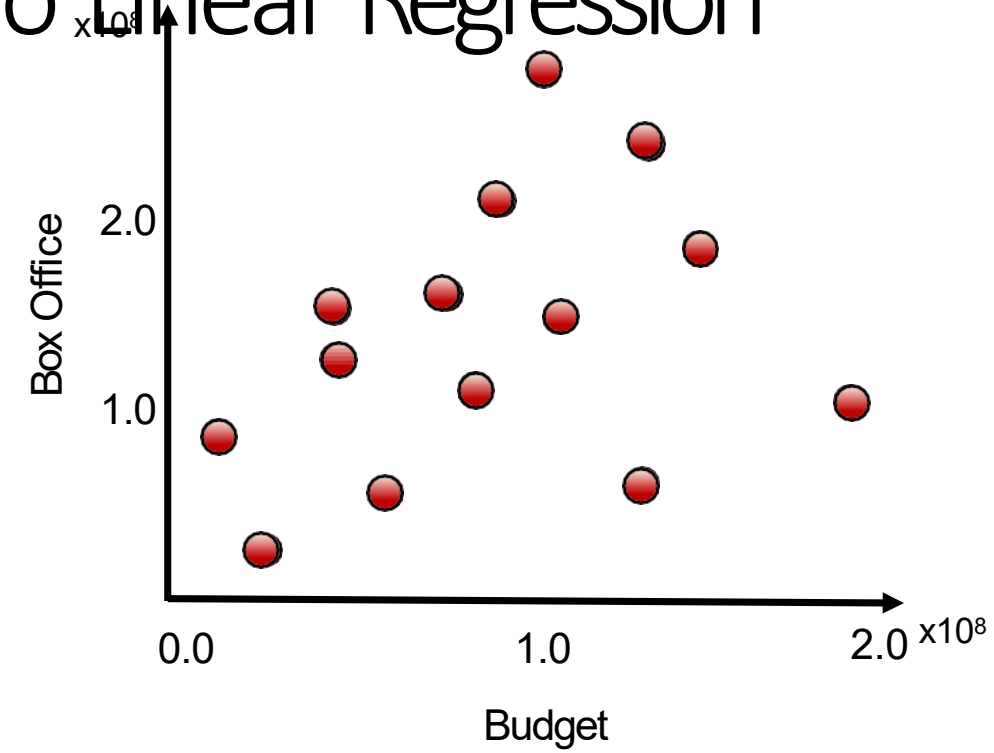
(AI)

Introduction to Linear Regression



$$y_\beta(x) = \beta_0 + \beta_1 x$$

Introduction to Linear Regression



$$y_\beta(x) = \beta_0 + \beta_1 x$$

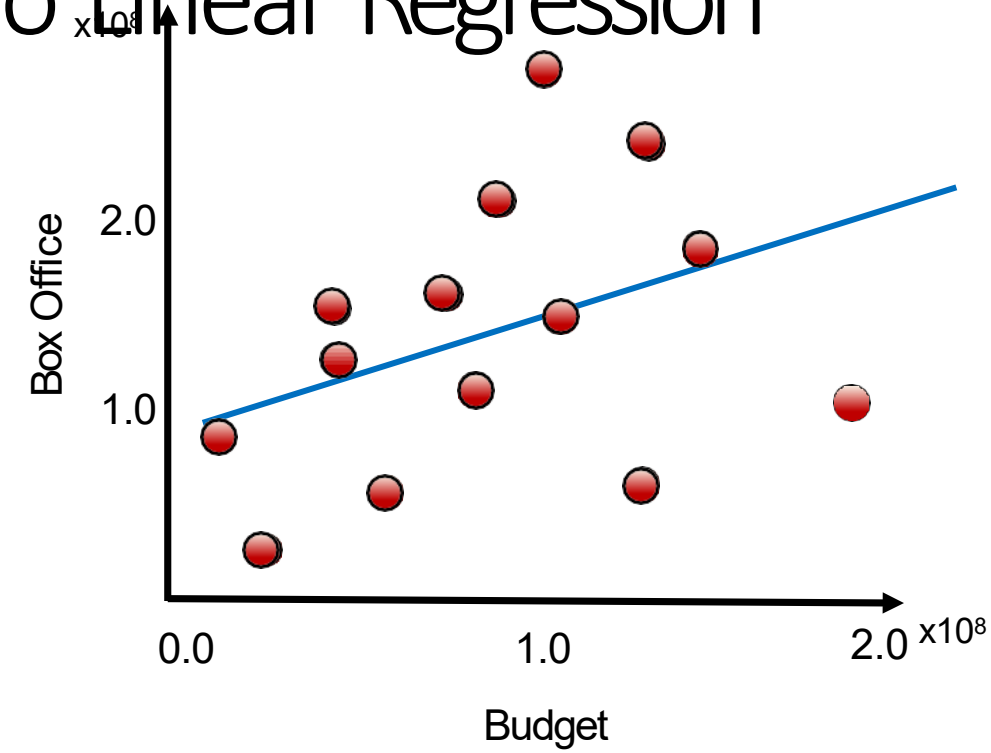
box office revenue

coefficient 0

coefficient 1

movie budget

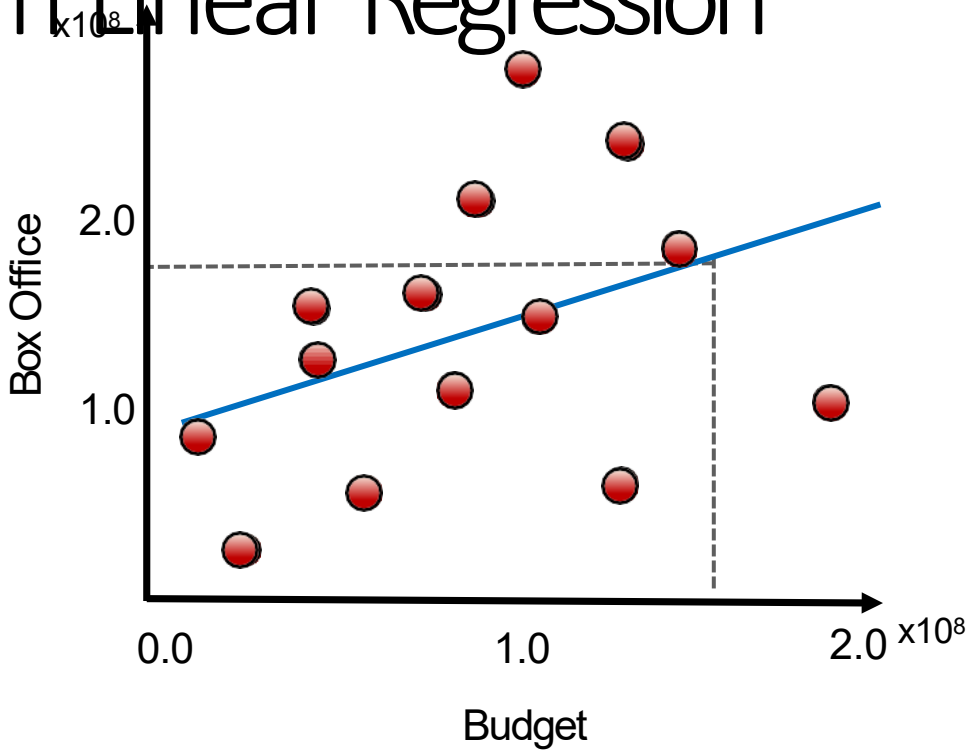
Introduction to Linear Regression



$$y_\beta(x) = \beta_0 + \beta_1 x$$

β_0= 80 million, β_1= 0.6

Predicting from Linear Regression

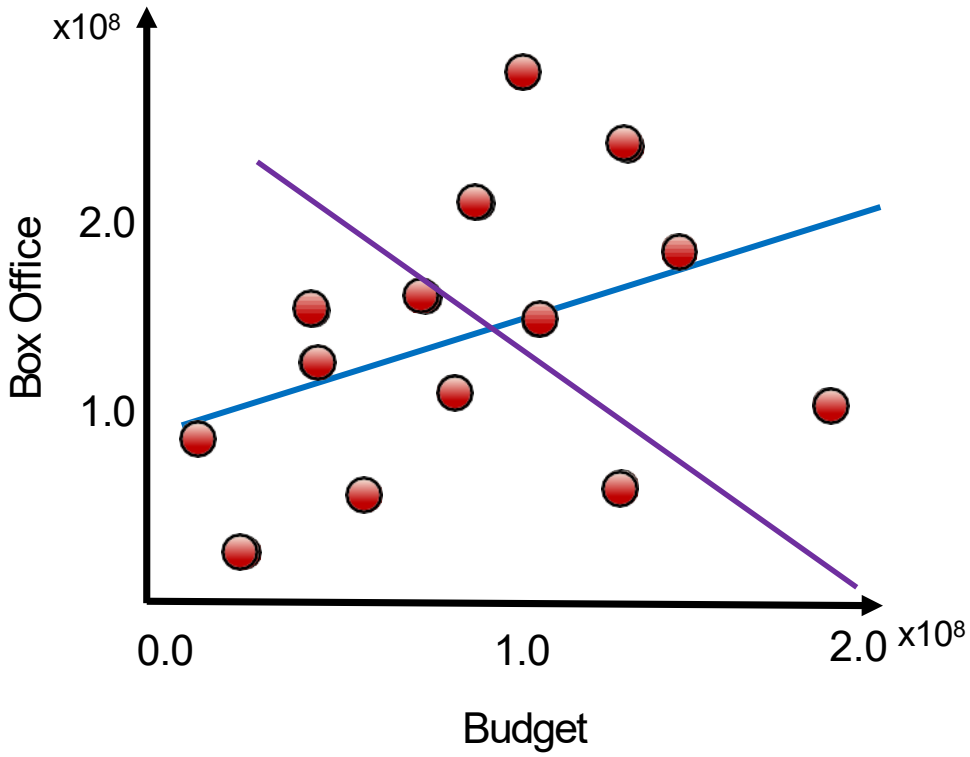


$$y_\beta(x) = \beta_0 + \beta_1 x$$

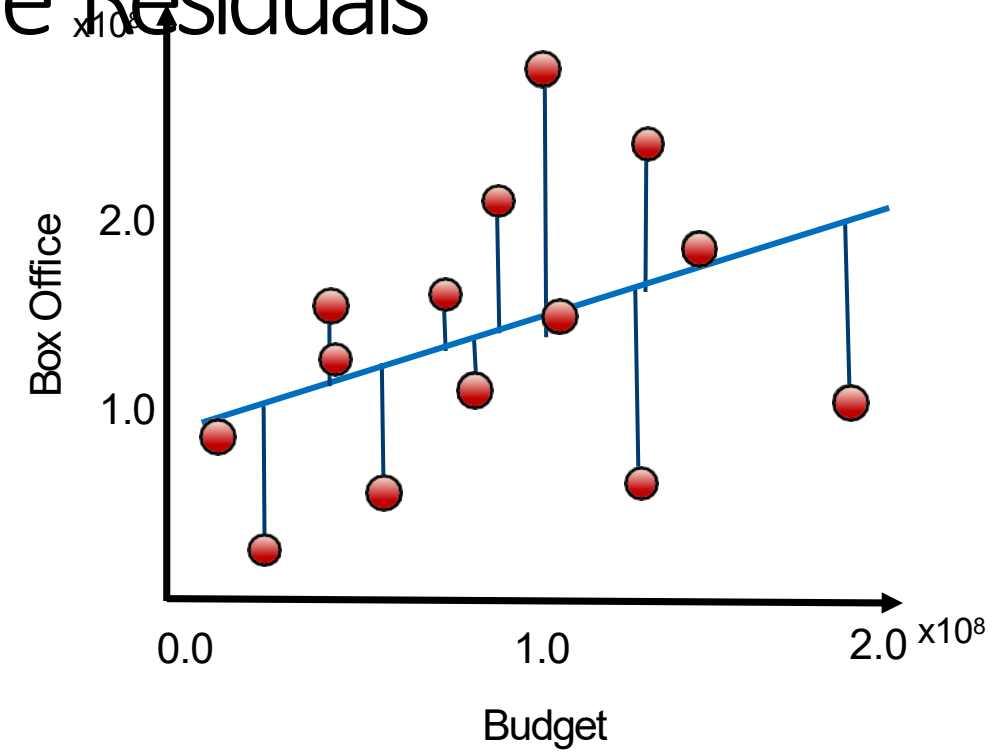
β_0= 80 million, β_1= 0.6

Predict 176Million Gross for 160 Million Budget

Which Model Fits the Best?



Calculating the Residuals

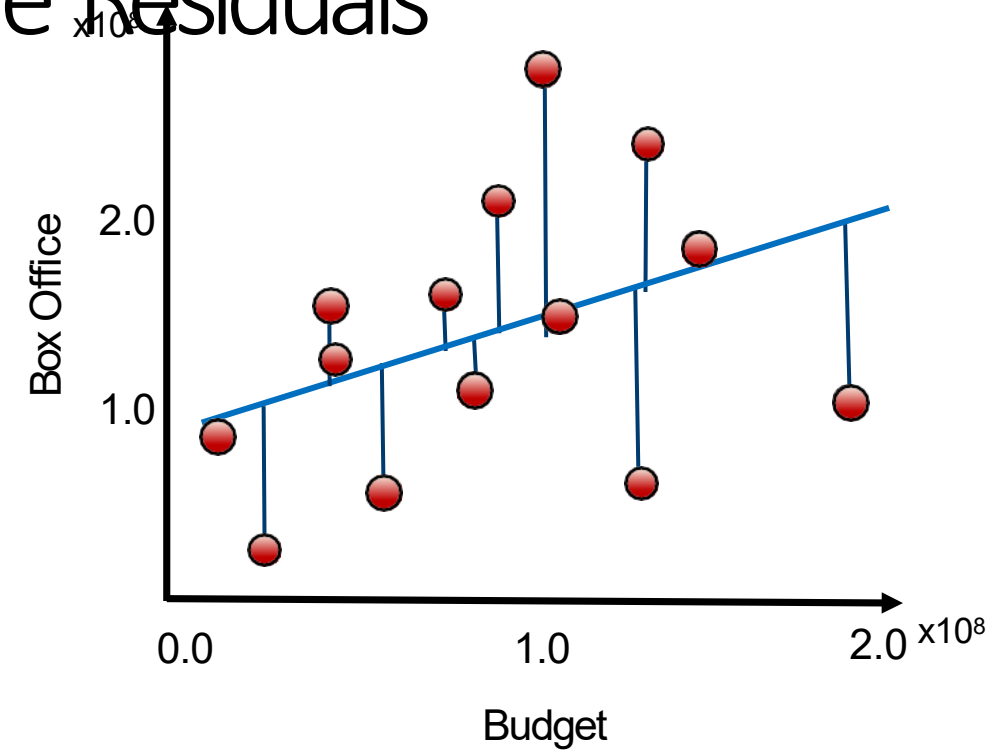


$$y_\beta\left(x_{obs}^{(i)}\right) - y_{obs}^{(i)}$$

predicted value

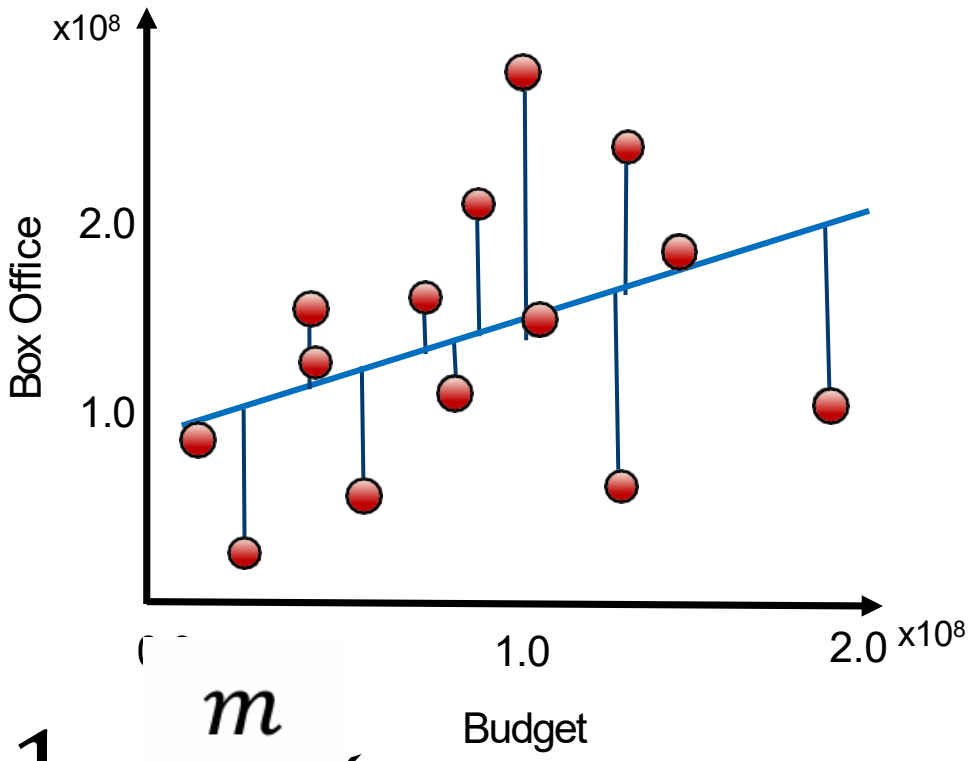
real value

Calculating the Residuals



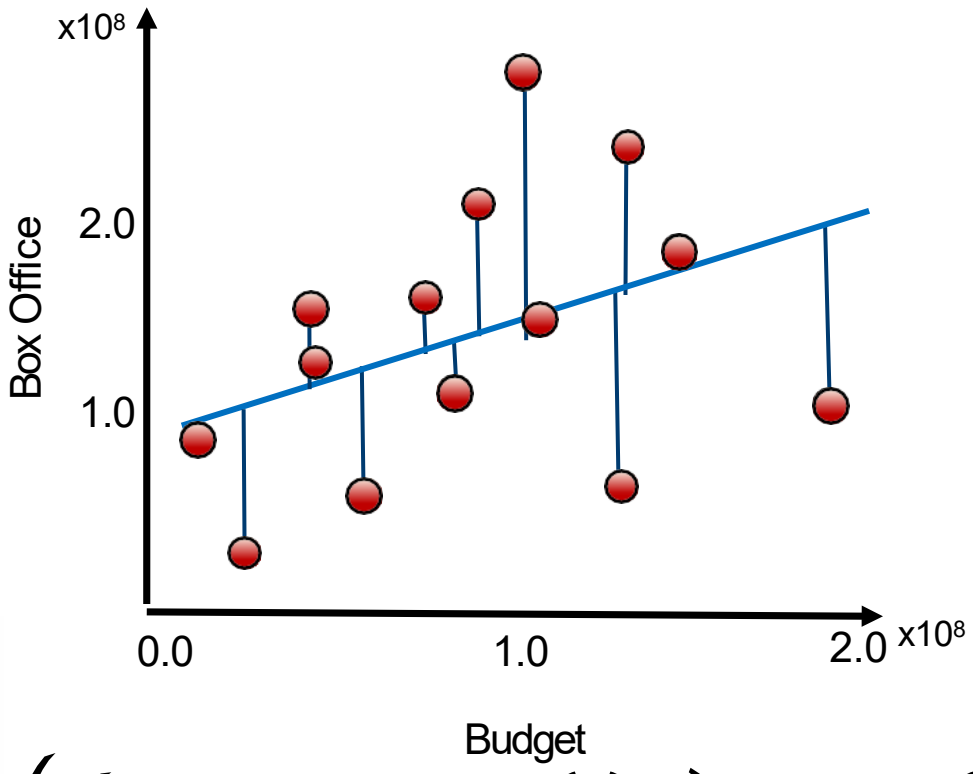
$$\left(\beta_0 + \beta_1 x_{obs}^{(i)}\right) - y_{obs}^{(i)}$$

Cost Function



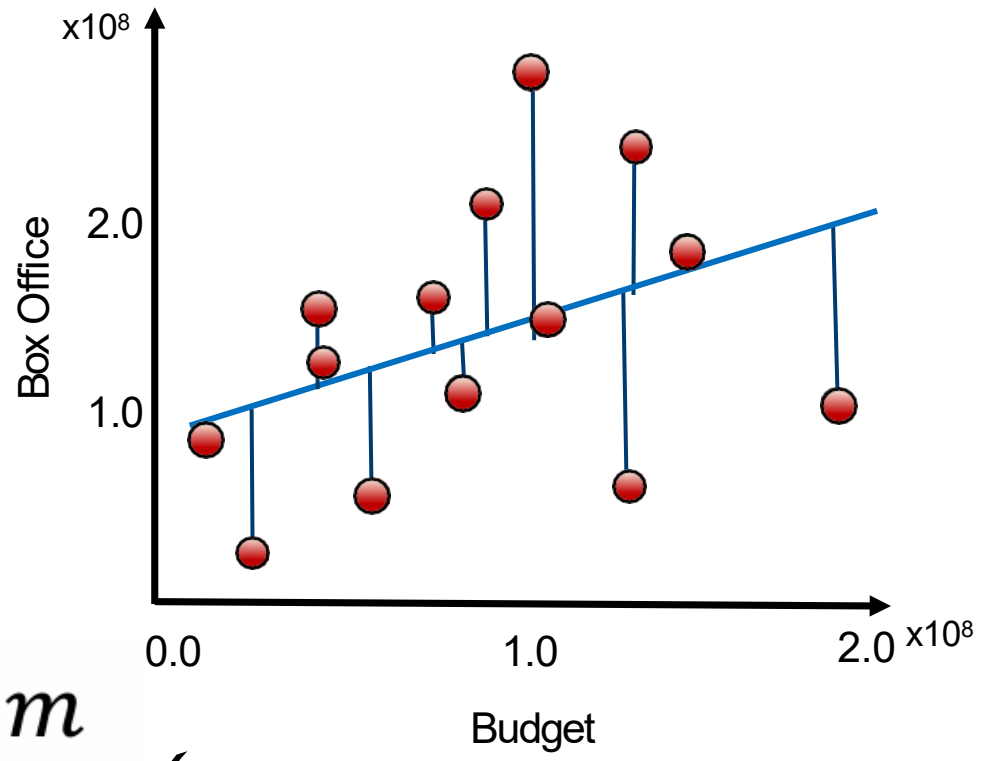
$$J(\beta_0, \beta_1) = \frac{1}{2m} \sum_{i=1}^{m} \left(\left(\beta_0 + \beta_1 x_{obs}^{(i)} \right) - y_{obs}^{(i)} \right)^2$$

Mean Squared Error



$$\frac{1}{m}\sum_{i=1}^{m}\left(\left(\beta_0 + \beta_1 x_{obs}^{(i)}\right) - y_{obs}^{(i)}\right)^2$$

Minimum Mean Squared Error

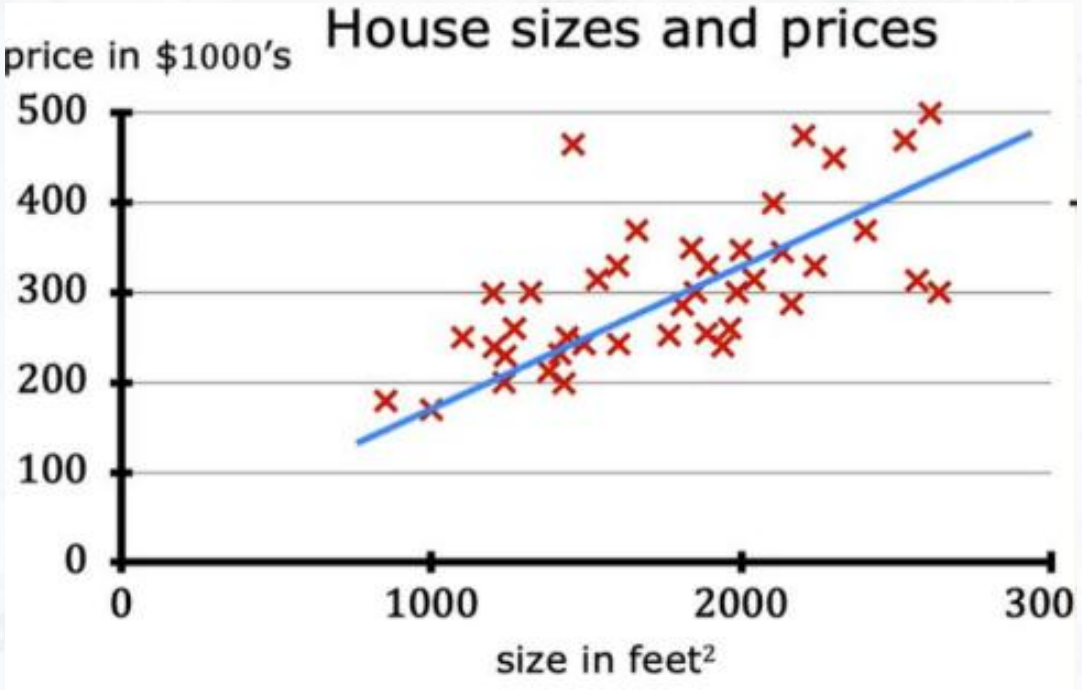


$$\min_{\beta_0,\beta_1} \frac{1}{m} \sum_{i=1}^{m} \left(\left(\beta_0 + \beta_1 x_{obs}^{(i)} \right) - y_{obs}^{(i)} \right)^2$$

LR equation

$$\hat{y} = \theta_0 + \theta_1 x_1$$

- $\hat{y}$ is the predicted value.
- x_1 is the feature value.
- θ_0 is the bias.
- θ_1 is the feature weight



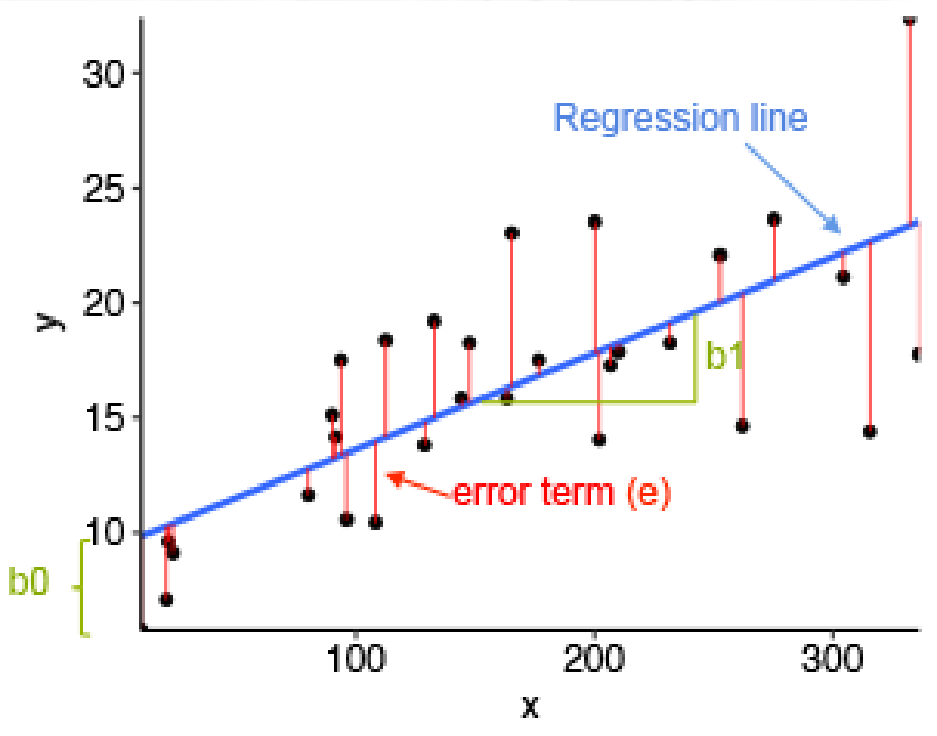
For multiple features:

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n$$

Error & Loss

- **Error:** The difference between the real value and the predicted value
- **Loss Function:** A function that quantifies the model's performance based on the prediction error.
- **MSE (Mean Squared Error):** N is number of samples:

$$MSE = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$$



Modelling Best Practice

- Use cost function to fit model
- Develop multiple models
- Compare results and choose best one

Comparing Linear Regression and KNN

Linear Regression

- Fitting involves minimizing cost function (*slow*)

K Nearest Neighbors

- Fitting involves storing training data (*fast*)

Comparing Linear Regression and KNN

Linear Regression

- Fitting involves minimizing cost function (*slow*)
- Model has few parameters (*memory efficient*)

K Nearest Neighbors

- Fitting involves storing training data (*fast*)
- Model has many parameters (*memory intensive*)

Comparing Linear Regression and KNN

Linear Regression

- Fitting involves minimizing cost function (*slow*)
- Model has few parameters (*memory efficient*)
- Prediction involves calculation (*fast*)

- K Nearest Neighbors

- Fitting involves storing training
- Model has many parameters (
  intensive)
- Prediction involves finding clos
- neighbors (*slow*)

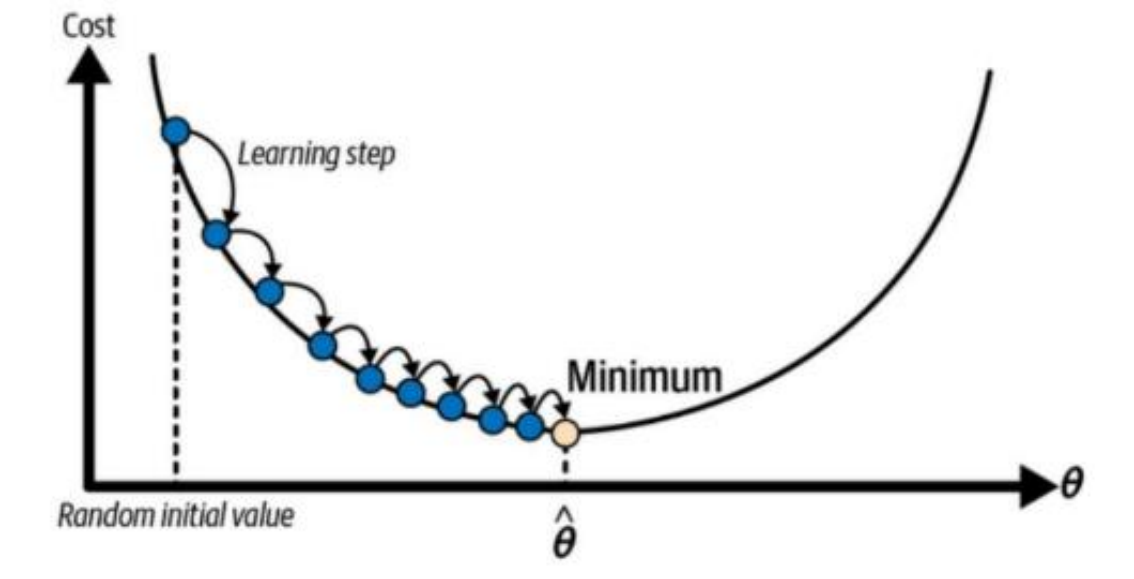
Gradient Descent

- **Optimization** algorithm to minimize loss

Repeat until convergence {

$$\theta_j \leftarrow \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta)$$

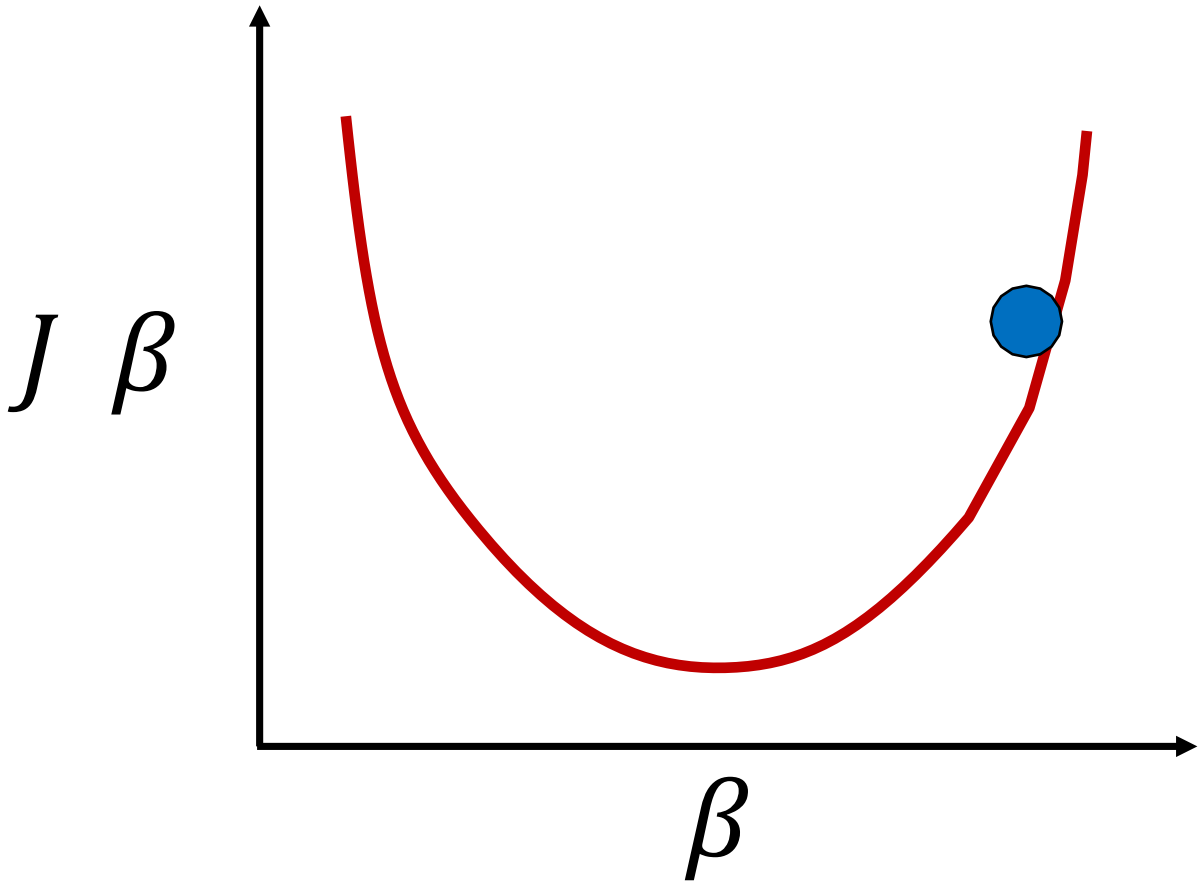
}



- **Convergence:** stop when MSE change is small or after max iterations
- **Learning Rate (α):** controls step size of updates

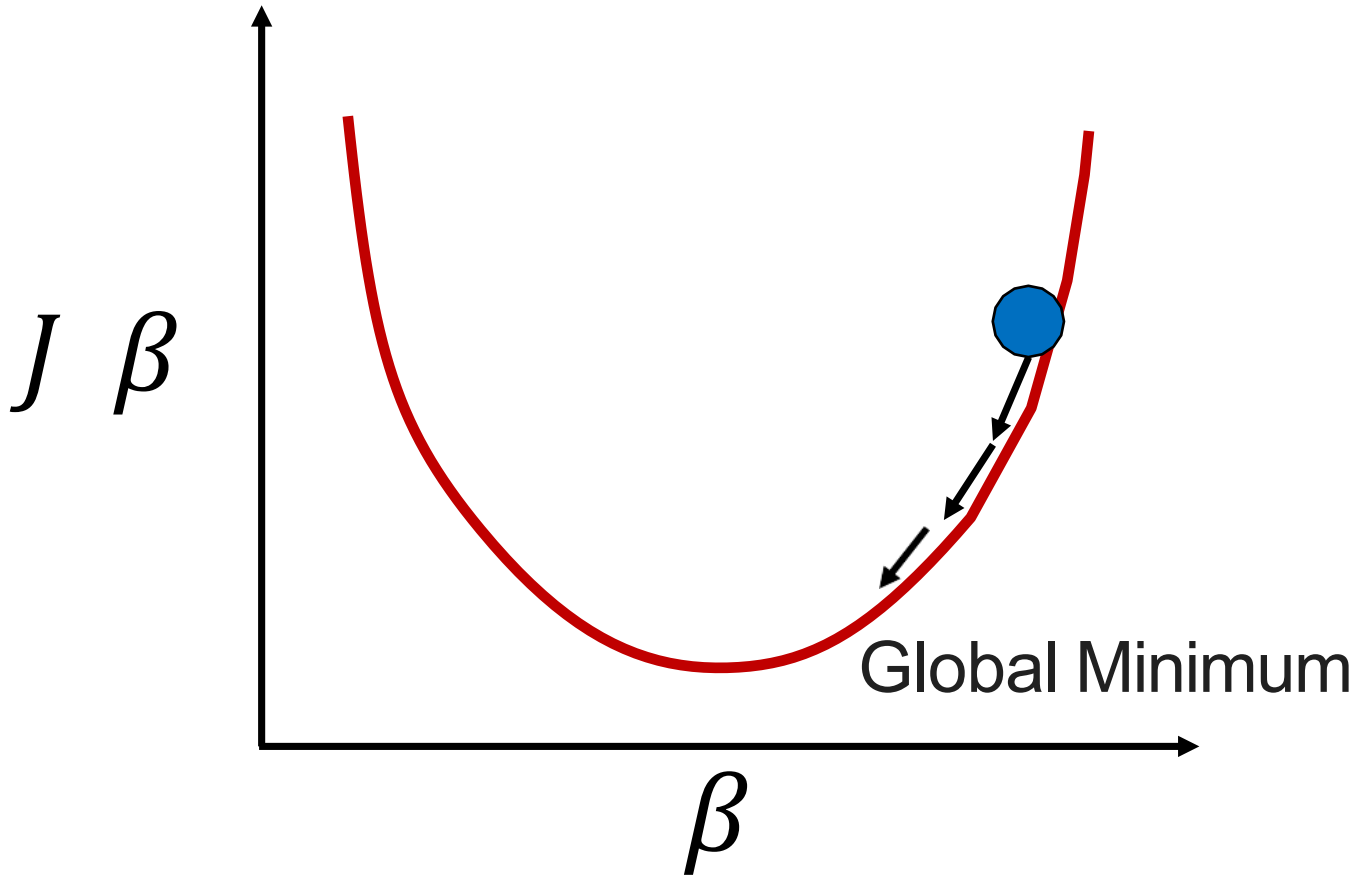
Gradient Descent

Start with a cost function J(β):



Gradient Descent

Start with a cost function J(β):



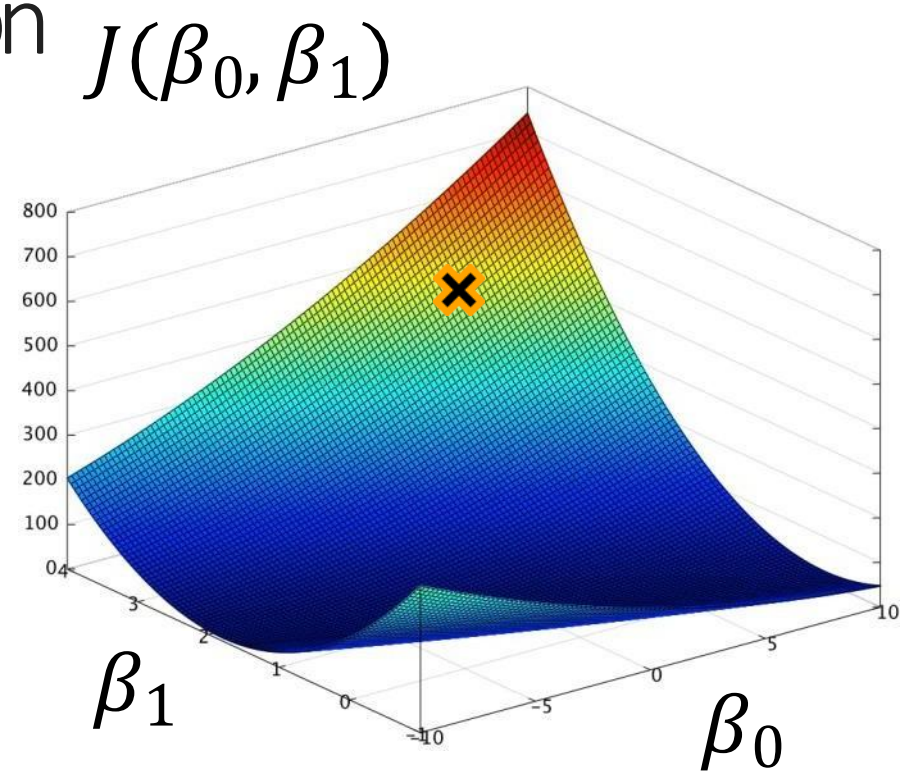
Then gradually move towards the minimum.

Gradient Descent with Linear Regressic

- Now imagine there are two parameters (β_0, β_1)

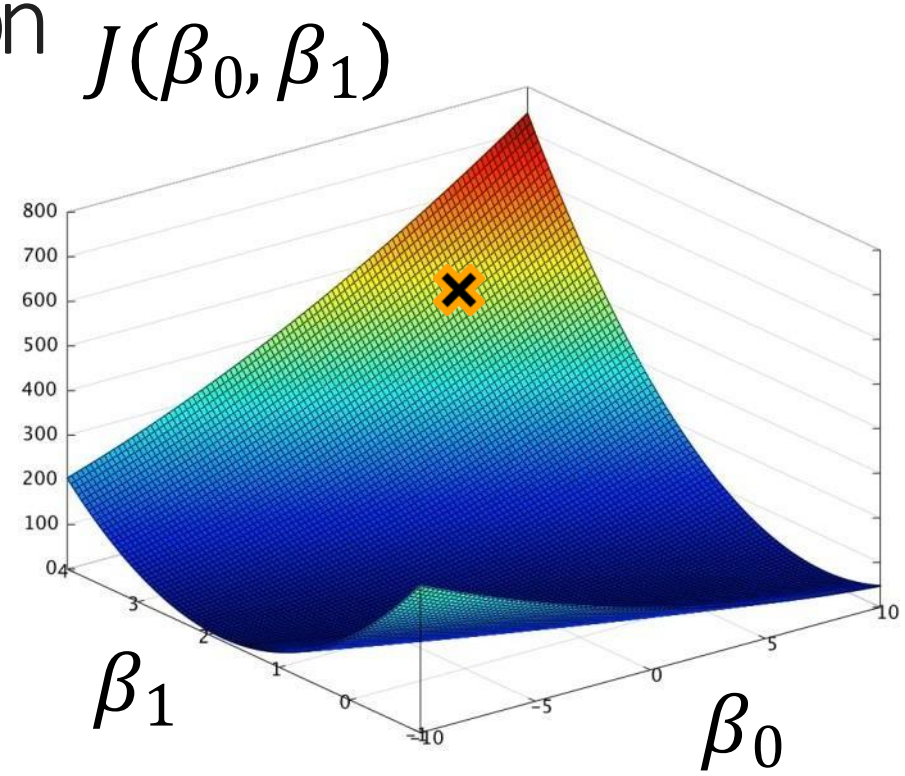
Gradient Descent with Linear Regression

- Now imagine there are two parameters (β_0, β_1)
- This is a more complicated surface on which the minimum must be found
- How can we do this without knowing what



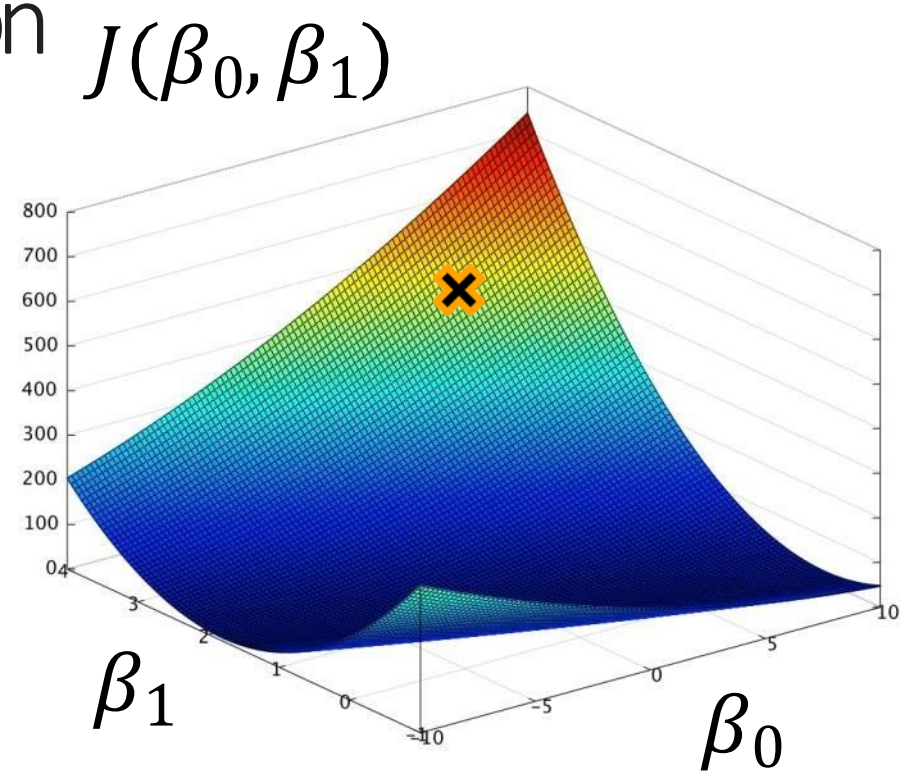
Gradient Descent with Linear Regression

- Now imagine there are two parameters (β_0, β_1)
- This is a more complicated surface on which the minimum must be found
- How can we do this without knowing what $J(\beta_0, \beta_1)$ looks like?



Gradient Descent with Linear Regression

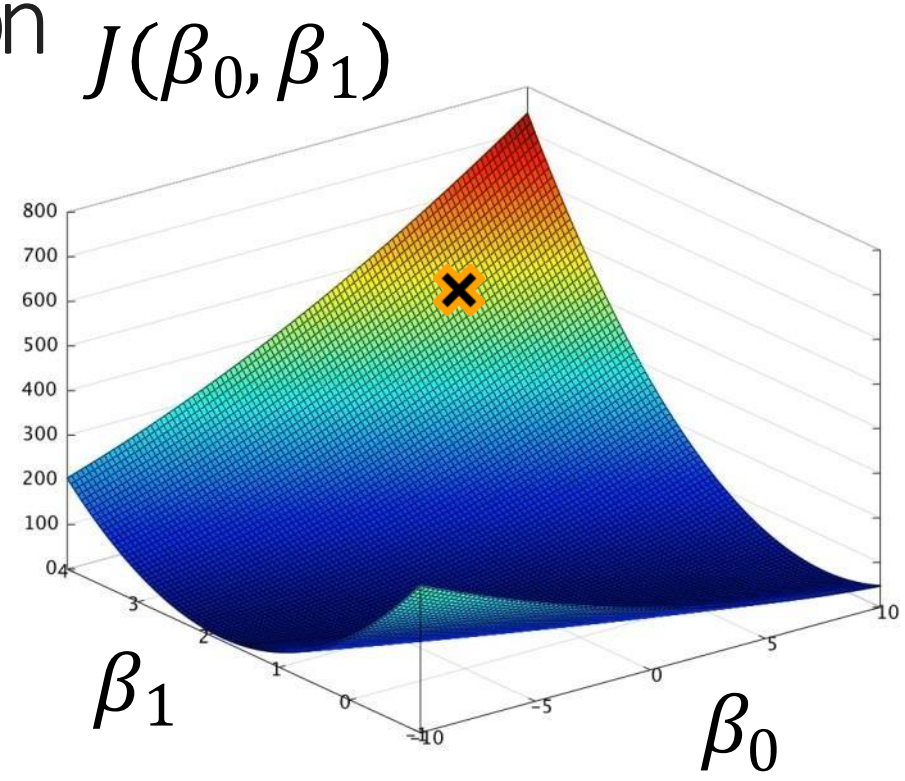
- Compute the gradient, $\nabla J(\beta_0, \beta_1)$, which points in the direction of the biggest increase!
- $-\nabla J(\beta_0, \beta_1)$ (negative gradient) points to the biggest decrease at that point!



Gradient Descent with Linear Regression

- The gradient is the a vector whose coordinates consist of the partial derivatives of the parameters

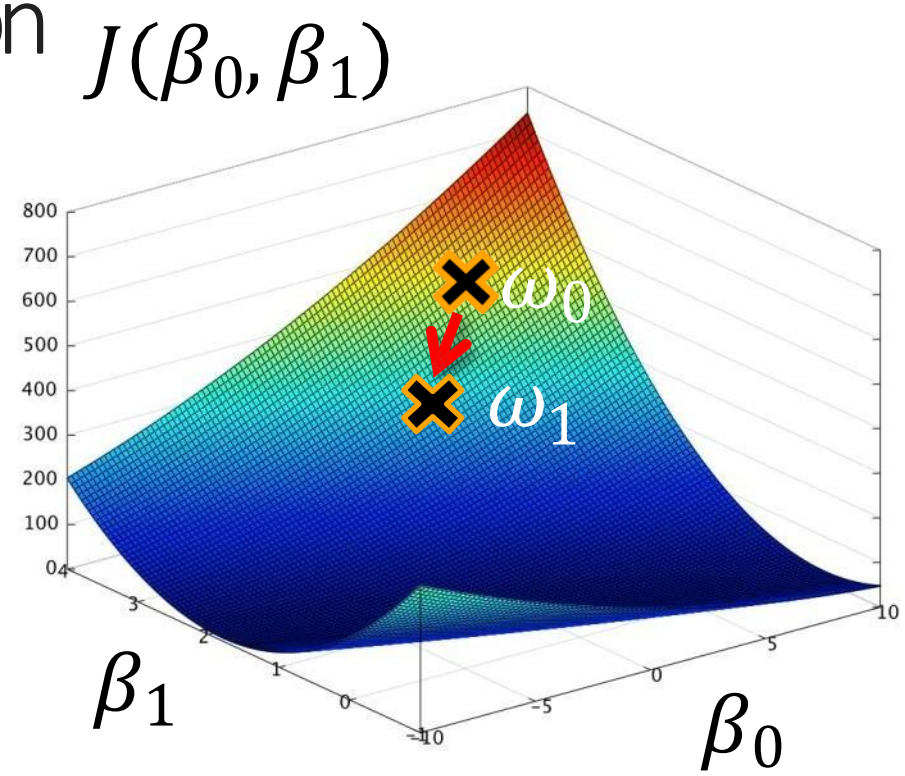
$$\nabla J(\beta_0, \dots, \beta_n) = < \frac{\partial J}{\partial \beta_0}, \dots, \frac{\partial J}{\partial \beta_n} >$$



Gradient Descent with Linear Regression

- Then use the gradient (∇) and the cost function to calculate the next point (ω_1) from the current one (ω_0):

$$\omega_1 = \omega_0 - \alpha \nabla \frac{1}{2} \sum_{i=1}^{m} \left(\left(\beta_0 + \beta_1 x_{obs}^{(i)} \right) - y_{obs}^{(i)} \right)^2$$

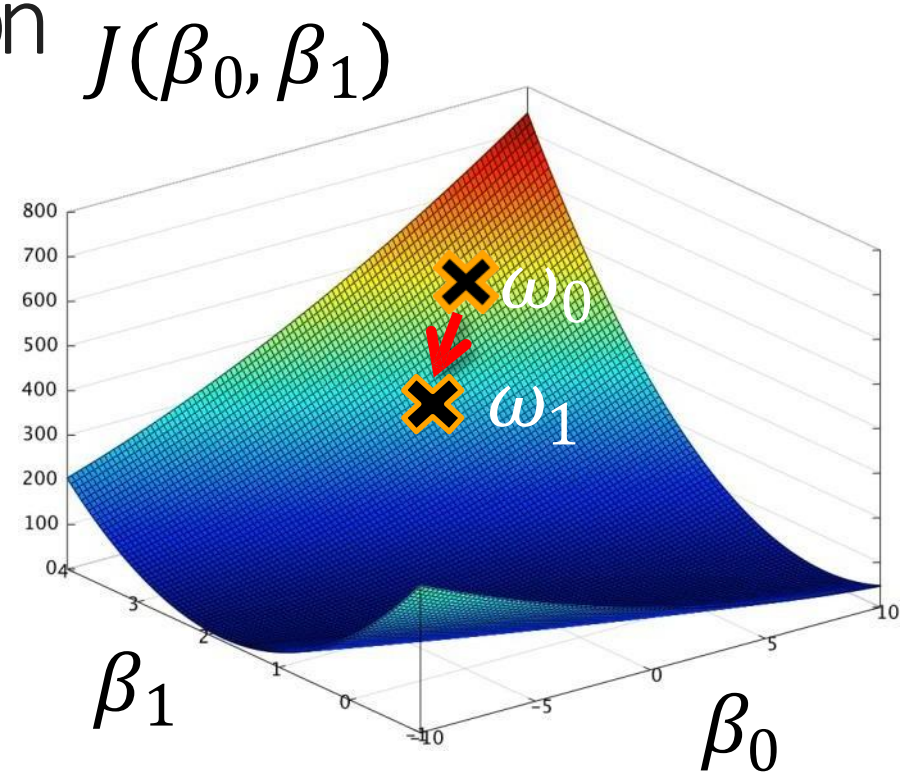


Gradient Descent with Linear Regression

- Then use the gradient (∇) and the cost function to calculate the next point (ω_1) from the current one (ω_0):

$$\omega_1 = \omega_0 - \alpha \nabla \frac{1}{2} \sum_{i=1}^{m} \left(\left(\beta_0 + \beta_1 x_{obs}^{(i)} \right) - y_{obs}^{(i)} \right)^2$$

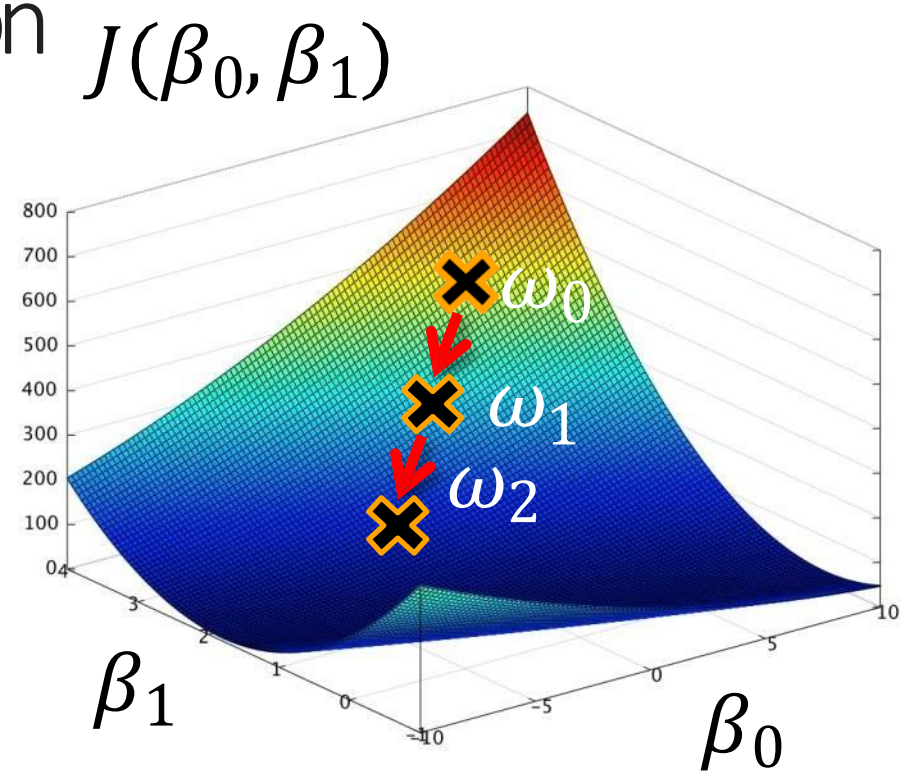
- The learning rate (α) is a tunable parameter that determines step size



Gradient Descent with Linear Regression

- Each point can be iteratively calculated from the previous one

$$\omega_2 = \omega_1 - \alpha \nabla \frac{1}{2} \sum_{i=1}^{m} \left(\left(\beta_0 + \beta_1 x_{obs}^{(i)} \right) - y_{obs}^{(i)} \right)^2$$

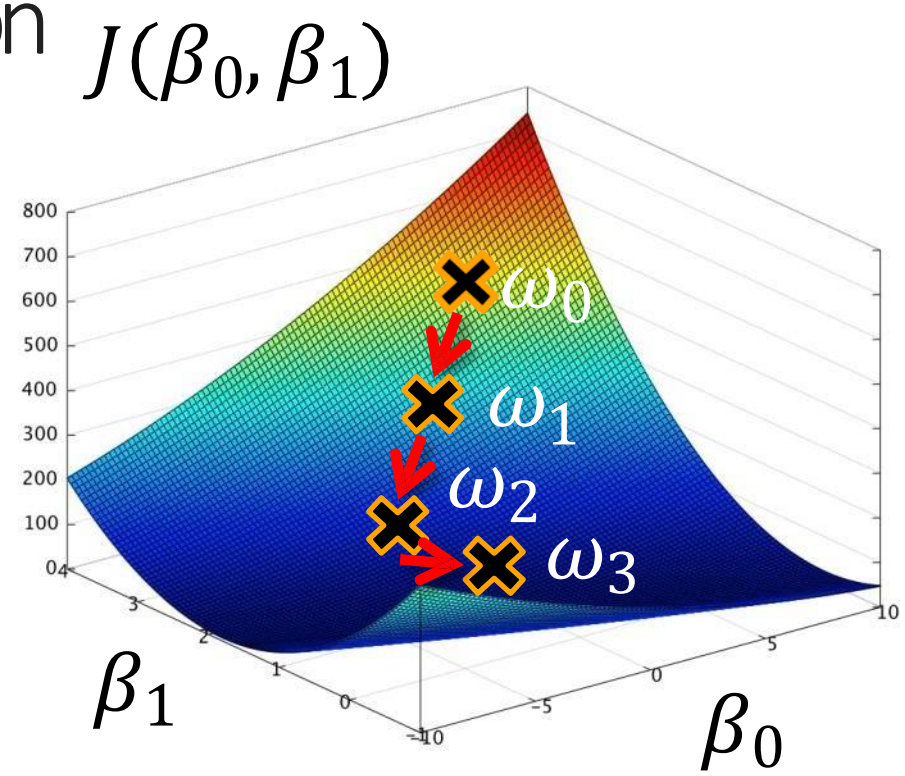


Gradient Descent with Linear Regression

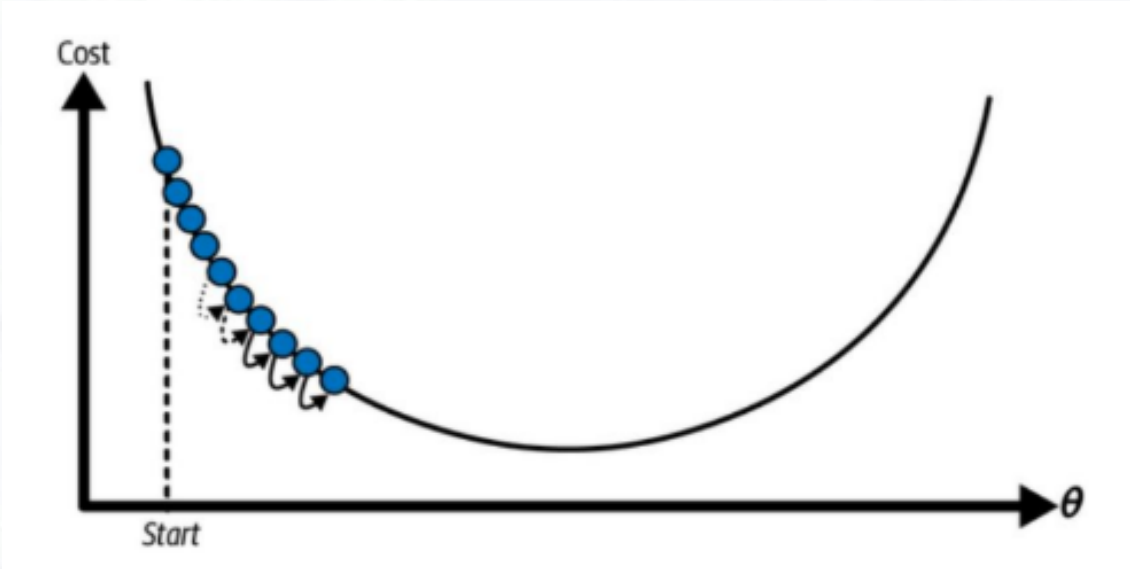
- Each point can be iteratively calculated from the previous one

$$\omega_2 = \omega_1 - \alpha\nabla\frac{1}{2}\sum_{i=1}^{m}\left(\left(\beta_0 + \beta_1 x_{obs}^{(i)}\right) - y_{obs}^{(i)}\right)^2$$

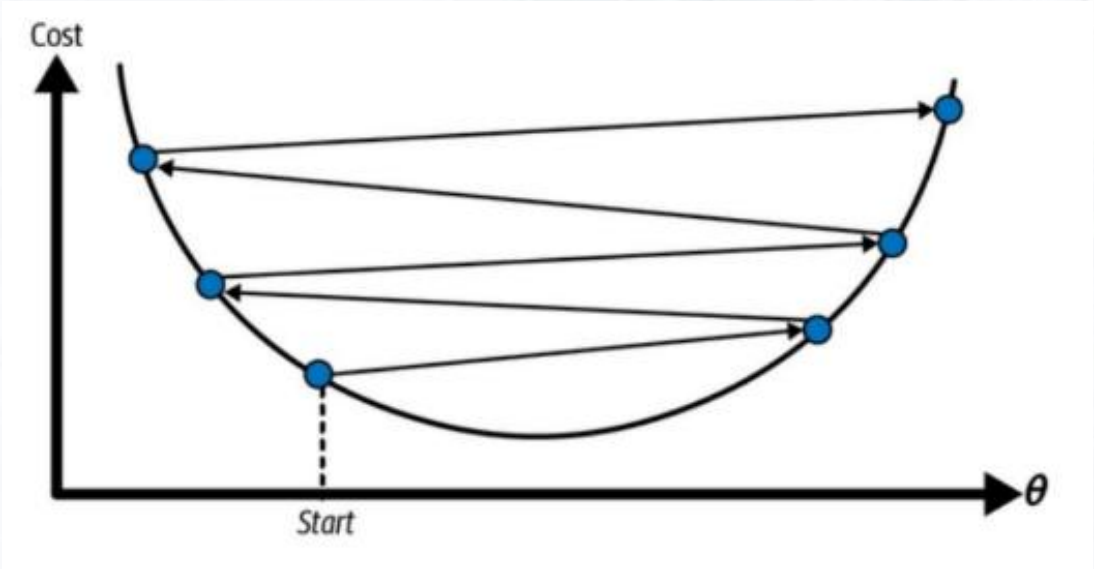
$$\omega_3 = \omega_2 - \alpha\nabla\frac{1}{2}\sum_{i=1}^{m}\left(\left(\beta_0 + \beta_1 x_{obs}^{(i)}\right) - y_{obs}^{(i)}\right)^2$$



Learning rate



Learning rate is too small



Learning rate is too high

Comparing Linear Regression and KNN

Linear Regression

- Fitting involves minimizing cost function (*slow*)

K Nearest Neighbors

- Fitting involves storing training data (*fast*)

Comparing Linear Regression and KNN

Linear Regression

- Fitting involves minimizing cost function (*slow*)
- Model has few parameters (*memory efficient*)

K Nearest Neighbors

- Fitting involves storing training data (*fast*)
- Model has many parameters (*memory intensive*)

Comparing Linear Regression and KNN

Linear Regression	• K Nearest Neighbors
• Fitting involves minimizing cost function (*slow*) • Model has few parameters (*memory efficient*) • Prediction involves calculation (*fast*)	• Fitting involves storing training data (*fast*) • Model has many parameters (*memory intensive*) • Prediction involves finding closest neighbors (*slow*)

Types of Supervised Learning

Classification outcome is a category

02 →

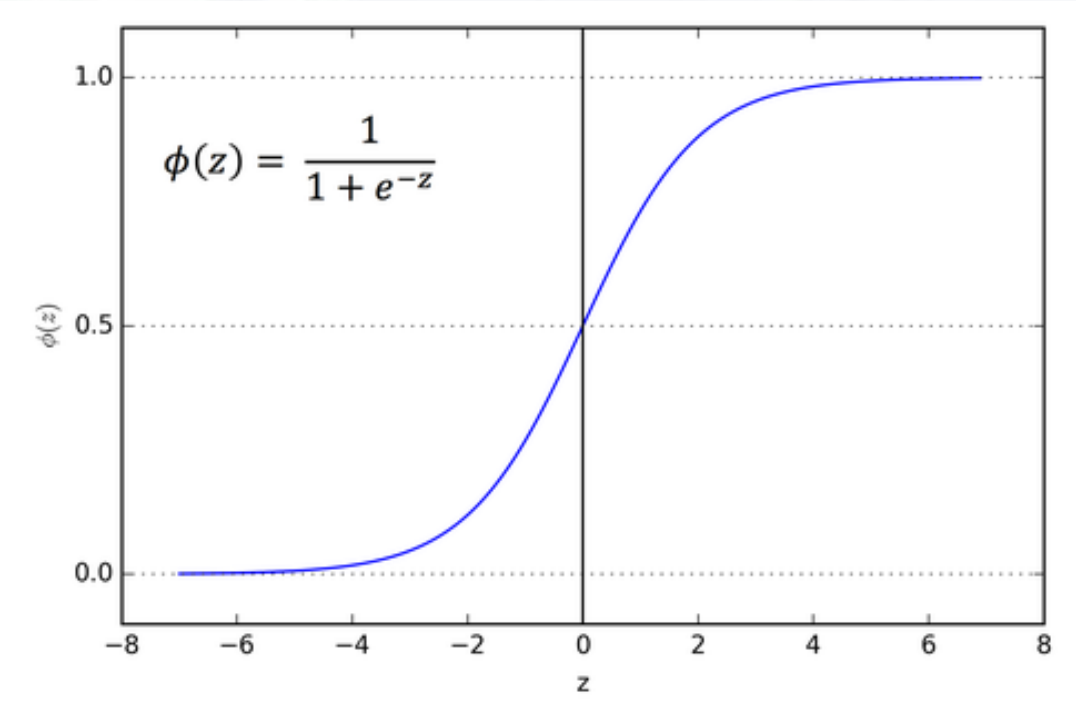
Logistic Regression

(AI)

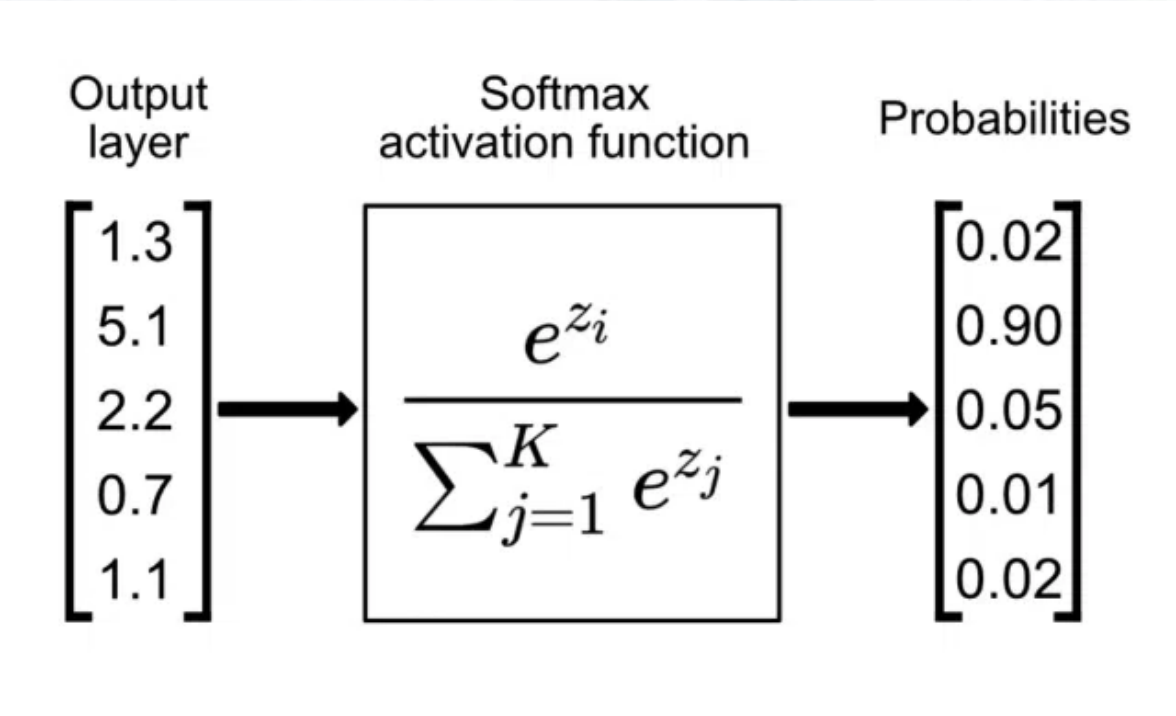
Logistic Regression

- Used for classification (binary or multiclass)
- Same linear function: **z = w*x + b**
- Instead of predicting directly, apply:
  - **Sigmoid:** for binary classification
  - **Softmax:** for multiclass classification

Sigmoid & Softmax



Sigmoid



Softmax

Loss function: Categorical Cross Entropy

$$L = -\frac{1}{n}\sum_{i=1}^{n}\sum_{k=1}^{K} y_k^{(i)} \log(\hat{y}_k^{(i)})$$

Variables:

- n: number of samples
- K: number of classes
- $y_k^{(i)}$: true label (1 if sample i belongs to class k, else 0)
- $\hat{y}_k^{(i)}$: predicted probability for class k for sample i

Loss function: Binary Cross Entropy

$$L = -\frac{1}{n}\sum_{i=1}^{n}\left[y^{(i)}\log(\hat{y}^{(i)}) + (1-y^{(i)})\log(1-\hat{y}^{(i)})\right]$$

Variables:

- n: number of samples
- $y^{(i)}$: true label for sample (i) (0 or 1)
- $\hat{y}^{(i)}$: predicted probability for the positive class for sample (i)